

MINDUEL LITE

An Interest-Based Two-Player Live-Audio Intellectual Competition Web Application

IU International University of Applied Sciences

Course: DLBSEPPSD01_E — Software Development

Examination: Oral Project Report · Task 2 — Development of a Web Application

GitHub repository (title-slide link)

<https://github.com/MahmoudBKanaan/MiduelLite>

Mahmoud B. Kanaan · Matriculation number: see IU myCampus

Outline — process over UI walkthrough

Presentation focus (IU oral project report)

1. Task definition — problem, objective, users, requirements
2. Planning & process — Kanban, iterative-incremental model
3. Architecture & data — C4 containers, ER model, matchmaking
4. Implementation — state machine, core logic, security choices
5. Software result — operational demo evidence (not a screen tour)
6. Testing & Docker — automated tests, reproducible run
7. Reflection — goal achievement, limits, lessons, outlook

1. Task definition — problem & objective

Problem

Short intellectual interaction is often blocked by registration, permanent profiles, and heavy multiplayer platforms. Text-only flows also remove natural spoken exchange for a brief two-player duel.

Objective (Task 2 + project)

Deliver a demonstrable full-stack SPA for IU Task 2 (React, Express, PostgreSQL, tests, Docker Compose, docs) with two-user interaction. Live spoken answers via managed LiveKit are part of this project's defined solution — not a separate IU examination mandate for audio/WebRTC.

1. Task definition — users & benefit

Primary user: person seeking a short, anonymous intellectual competition without permanent identity or PII.

Derived requirements for the solution

- Zero signup friction → temporary session (playerId + sessionToken)
- Meaningful pairing → interest overlap matchmaking (no ML stack)
- Spoken two-user interaction → LiveKit room + turn-based peer scoring
- Demonstrable full stack → SPA + API + DB + Docker Compose (+ LiveKit Cloud for audio)

1. Task definition — MVP scope

Scope reduction is an engineering decision aligned with the knowledge base and IU Task 2.

In scope (built)	Explicitly excluded
React SPA, Express API, PostgreSQL	Accounts / OAuth / JWT login
Live spoken answers (managed LiveKit)	Self-hosted WebRTC / TURN stack
Matchmaking 3→2→1; answer-complete	Chat, rankings, history UI
Peer scoring, flags, results	Recording / transcription
Tests, Docker Compose (3 services), docs	K8s / microservices / AI scoring

2. Planning & process — Kanban

Model: Iterative-Incremental Development managed with Kanban

- Why not pure Scrum? Solo student, fixed MVP, short timeline — ceremonies add overhead without team coordination benefit.
- Increments: concept → foundation → matchmaking → match → result → LIVE AUDIO → tests/docs → presentation.
- Board columns: BACKLOG → READY → IN PROGRESS → TEST → DONE (evidence below).

```
196 — Kanban / LIVE AUDIO Increment

# Manual Lite — Manual Board
# Incremental Iterative-Incremental Development managed with Kanban
# Kanban board columns: BACKLOG, READY, IN PROGRESS, TEST, DONE
# Kanban board items: task, status, priority, assignee, due date, etc.
# Kanban board actions: move, create, delete, update, etc.
# Kanban board filters: by column, by status, by priority, etc.
# Kanban board settings: board name, columns, items, filters, etc.

## BACKLOG
1. [task] [status] [priority] [assignee] [due date]
2. [task] [status] [priority] [assignee] [due date]
3. [task] [status] [priority] [assignee] [due date]
4. [task] [status] [priority] [assignee] [due date]
5. [task] [status] [priority] [assignee] [due date]

## READY
1. [task] [status] [priority] [assignee] [due date]
2. [task] [status] [priority] [assignee] [due date]
3. [task] [status] [priority] [assignee] [due date]
4. [task] [status] [priority] [assignee] [due date]
5. [task] [status] [priority] [assignee] [due date]

## IN PROGRESS
1. [task] [status] [priority] [assignee] [due date]
2. [task] [status] [priority] [assignee] [due date]
3. [task] [status] [priority] [assignee] [due date]
4. [task] [status] [priority] [assignee] [due date]
5. [task] [status] [priority] [assignee] [due date]

## TEST
1. [task] [status] [priority] [assignee] [due date]
2. [task] [status] [priority] [assignee] [due date]
3. [task] [status] [priority] [assignee] [due date]
4. [task] [status] [priority] [assignee] [due date]
5. [task] [status] [priority] [assignee] [due date]

## DONE
1. [task] [status] [priority] [assignee] [due date]
2. [task] [status] [priority] [assignee] [due date]
3. [task] [status] [priority] [assignee] [due date]
4. [task] [status] [priority] [assignee] [due date]
5. [task] [status] [priority] [assignee] [due date]
```

Figure 1. Kanban board snapshot (own project process evidence).

2. Planning — user journey (spoken live audio)

Welcome → Pool → match created → audio connected → spoken answers → Result

Play again → Pool (new match + new LiveKit room) · Reset → Welcome

Minduel Lite

Anonymous two-player intellectual competition



Display name
Neo

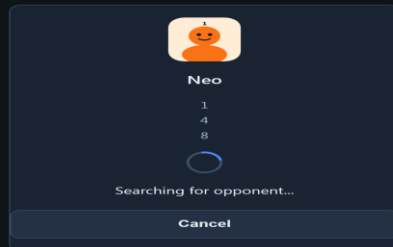
Avatar (choose one of 12)

Choose exactly 3 interests
Selected 3 / 3

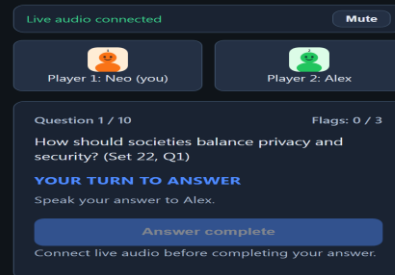
Technology Science Philosophy

Enter pool

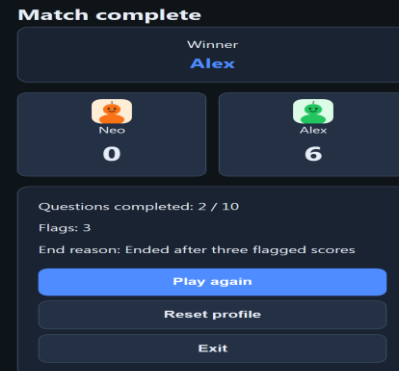
Welcome



Pool



Match + audio

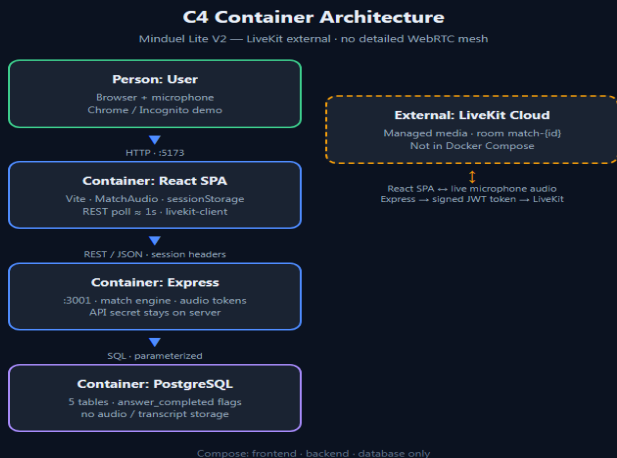


Result

Figures 2–5. Four-screen journey with live spoken match (own software).

3. Architecture — C4 + LiveKit

Simple distinction: Express + PostgreSQL = game state · LiveKit = live audio · REST polling = game sync



Talking points

- EXPRESS + POSTGRESQL
= authoritative game state
- LIVEKIT (managed Cloud)
= live microphone audio
- REST polling ≈ 1s
= game synchronisation
- LiveKit is part of the
project's defined solution
(not an IU audio mandate)

Figure 6. C4 + external LiveKit (own diagram). Media API: LiveKit docs (external).

3. Architecture — data model

ER diagram — match_rounds (spoken turns)

No LiveKit entities — audio is not persisted

```
players 1 —0..1 queue_entries
      ↓
      as player1 / player2
matches 1 —1..10— match_rounds
      ↓
      competition_id (logical)
questions (competition_id, question_number, question_text)

match_rounds columns:
  match_id, question_number
  player1_answer_completed BOOLEAN
  player1_score, player1_score_flagged, player1_reviewed
  player2_answer_completed BOOLEAN
  player2_score, player2_score_flagged, player2_reviewed

REMOVED: player1_answer TEXT · player2_answer TEXT
NOT IN ER: LiveKit rooms, recordings, transcripts
```

Spoken audio is transient via LiveKit only. Completion flags mark that a player finished speaking.

Figure 7. ER (own). *answer_completed* booleans; no answer TEXT; LiveKit not in ER (audio not persisted).

4. Implementation — LiveKit token & MatchAudio

audioTokenService.js — signed grant (secret on server only)

```
const at = new AccessToken(apiKey, apiSecret, {
  identity: playerId,
  ttl: '2h',
});
at.addGrant({
  roomJoin: true,
  room: `match-${matchId}`,
  canPublish: true,
  canSubscribe: true,
  canPublishSources: [MICROPHONE],
  canPublishData: false,
});
const token = await at.toJwt();
return { token, serverUrl };
```

MatchAudio.jsx — connect lifecycle

```
// CONNECTING → getAudioToken → Room
const { token, serverUrl } =
  await getAudioToken(matchId);
room = new Room({ ... });
room.on(TrackSubscribed, attachRemote);
await room.connect(serverUrl, token);
await room.localParticipant
  .setMicrophoneEnabled(true);
setAudioState('CONNECTED');

// cleanup on unmount only:
// mic off, disconnect, remove listeners
// deps: [matchId] — not phase/question
```

Actual source excerpts. Membership validated before token; no API secret in the browser.

4. Implementation — matchmaking & third flag

matchmakingService.js

```
export function selectBestCandidate(
  currentInterests, waitingPlayers) {
  return waitingPlayers
    .map(c => ({ ...c,
      overlap: calculateInterestOverlap(
        currentInterests, c.interests) })))
    .filter(c => c.overlap >= 1) // never 0
    .sort((a, b) => {
      if (b.overlap !== a.overlap)
        return b.overlap - a.overlap;
      return new Date(a.joined_at)
        - new Date(b.joined_at);
    })[0] || null;
}
```

matchService.js — submitReview

```
if (flag) {
  await client.query(
    `UPDATE matches
      SET flag_count = flag_count + 1
      WHERE id = $1`, [matchId]);
}
const m = (await client.query(
  'SELECT * FROM matches WHERE id = $1',
  [matchId])).rows[0];

// Immediate end — do not wait for
// the other player's review
if (m.flag_count >= 3) {
  await client.query(
    `UPDATE matches SET status='ENDED',
      end_reason='THREE_FLAGS',
      ended_at=NOW() WHERE id=$1`,
    [matchId]);
}
```

Business logic excerpts: interest priority 3→2→1; third flag ends at count 3 (parameterized SQL).

5. Testing & reproducible run

Backend: audio-token (participant / outsider / ended match / no secret leak); answer-complete; third-flag from flag_count=2

Frontend: MatchAudio CONNECTING / CONNECTED / FAILED + RETRY; cleanup disconnect; Answer complete calls completeAnswer

Manual: Chrome + Incognito; mic permission; Live audio connected; speak both ways; full spoken match path

210 – Backend automated tests (V2)

```
> minduel-lite-backend@1.0.0 test
> vitest run

 RUN v2.1.9 0:/Documents/EU/Minduelite/backend

 ✓ tests/core-mvp.test.js (17 tests) 2573ms
 ✓ BT core - match rules > third-flag termination 1526ms
 ✓ tests/api.test.js (10 tests) 1000ms
 ✓ Player and matchmaking (DB) > BT-10/11 flagging and third flag ends match 362ms
 ✓ tests/match-engine.test.js (12 tests) 111ms
 ✓ third flag from flag_count = 2 > when flag_count is 2, one player flag=true ends match immediately (THREE_FLAGS) 317ms
 ✓ tests/review.test.js (10 tests) 820ms
 ✓ tests/result.test.js (9 tests) 347ms
 ✓ GET /api/matches/{matchid}/result (items 68-69) > returns result payload after THREE_FLAGS end 341ms
 ✓ tests/audio-token.test.js (6 tests) 1040ms
 ✓ tests/matchmaking.test.js (13 tests) 440ms
 ✓ tests/security.test.js (11 tests) 150ms
 ✓ tests/players.test.js (12 tests) 142ms
 ✓ tests/foundation.test.js (3 tests) 58ms

Test Files 10 passed (10)
Tests 189 passed (189)
Start at 19:51:29
Duration 10.68s (transform 187ms, setup 0ms, collect 7.68s, tests 7.70s, environment 5ms, prepare 1.73s)
```

212 – Docker containers running (V2)

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
minduelite-backend-1	minduelite-backend	"docker-entrypoint.s..."	backend	9 minutes ago	Up 9 minutes	0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
minduelite-database-1	postgres:16-alpine	"docker-entrypoint.s..."	database	9 minutes ago	Up 9 minutes (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
minduelite-frontend-1	minduelite-frontend	"docker-entrypoint.s..."	frontend	9 minutes ago	Up 9 minutes	0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp

docker compose up --build · LiveKit env on backend only

Figures. Backend test evidence; Docker services (own environment).

5. Software result — what was demonstrated

Evidence of an operational system (process outcome), not a feature-by-feature UI description:

- Two browser sessions share Express + PostgreSQL; each joins LiveKit for speech
- Match created → audio connected → spoken answers (answer-complete, no text upload)
- Authoritative phases: speak → score → review; third flag ends immediately
- Speech not recorded/stored/transcribed; only completion flags + scores in DB
- Play-again: new match + new LiveKit room; Docker Compose runs three app services
- Repository: frontend, backend, DB, Docker, tests, docs (GitHub on title slide)

6. Evaluation — was the goal achieved?

Yes — for the defined academic MVP (not production readiness).

Evidence

- Two anonymous browser sessions
- Interest matchmaking (3→2→1)
- Live spoken turns + peer scoring
- Flags → THREE_FLAGS at count 3
- SPA + API + Postgres + LiveKit
- Docker Compose (3 app services)
- Automated tests pass (FE + BE)

Why LiveKit (not raw WebRTC)

LiveKit is part of this project's defined solution (not an IU-required audio stack). Managed LiveKit reduced:

- signaling complexity
- TURN/STUN responsibility
- connection-management complexity
- development time

(Technical API: LiveKit official docs.)

Limits: Cloud + internet, demo reconnection, no recording/transcription/moderation, temporary sessions, polling for game state.

6. Reflection & future improvements

Lessons from this implementation

- Managed LiveKit avoided raw WebRTC infrastructure
- Server-owned game state + short-lived media tokens
- Polling adequate for game sync; LiveKit for audio only
- Focused tests (token, audio states, third flag) beat volume

Discussed only (not implemented)

- Authenticated accounts & history
- WebSockets for game state; advanced reconnect
- Recording / transcription / voice moderation
- Cloud deploy of the app stack; rankings

Conclusion

- Task definition drove a deliberately small MVP
- Kanban + iterative increments structured solo development
- Architecture and state machine kept multiplayer consistent
- Tests, Docker, and GitHub complete the Task 2 evidence chain

<https://github.com/MahmoudBKanaan/MiduelLite>

List of figures

All figures below are the author's own work produced for this project (UI screenshots, process evidence, and original diagrams). External materials are listed only in the bibliography.

Figure 1. Kanban board snapshot (project process evidence).

Figures 2–5. Application screenshots: Welcome, Pool, Match, Result (own software).

Figure 6. C4 container architecture diagram (own diagram).

Figure 7. Entity-relationship diagram (own diagram).

Figure 8. User flow and match state machine (own diagram).

Figure 9. Automated test run output (own development environment).

Figure 10. Docker Compose services status (own development environment).

Bibliography

APA 7th-style references for external sources used in the project and presentation. Original project diagrams and screenshots are not external sources.

IU International University of Applied Sciences. (n.d.). Task for course DLBSEPPSD01_E – Software Development [Examination task]. Examination Office.

IU International University of Applied Sciences. (n.d.). Guidelines for the creation of an oral project report. Examination Office.

Meta Platforms. (n.d.). React documentation. <https://react.dev/>

OpenJS Foundation. (n.d.). Express — Node.js web application framework. <https://expressjs.com/>

PostgreSQL Global Development Group. (n.d.). PostgreSQL documentation. <https://www.postgresql.org/docs/>

Docker Inc. (n.d.). Docker Compose documentation. <https://docs.docker.com/compose/>

LiveKit. (n.d.). LiveKit documentation. <https://docs.livekit.io/>

LiveKit. (n.d.). Access tokens. <https://docs.livekit.io/home/server/generating-tokens/>

LiveKit. (n.d.). JavaScript client SDK. <https://docs.livekit.io/home/client/connect/>

Kanaan, M. B. (2026). Minduel Lite [Computer software]. GitHub. <https://github.com/MahmoudBKanaan/MiduelLite>